Task's Allocator Use

Document #:	D3980R0 [Latest] [Status]
Date:	2026-02-22
Project:	Programming Language C++
Audience:	Library Evolution Working Group (LEWG) Library Working Group (LWG)

Reply-to:	Dietmar Kühl (Bloomberg) <dkuhl@bloomberg.net>

Contents

There are different uses of allocators for `task`. The obvious use is that the coroutine frame needs to be allocated and using an allocator control where this coroutine frame gets allocated. In addition, the environment used when `connect`ing a sender can provide access to an allocator via the `get_allocator` query. The current specification uses the same allocator for coroutine frame and the child environments. At the Kona meeting the room preferred if these were separated and the allocator for the environment were taken from the environment of the receiver it gets `connect`ed to. In doing so, the allocator for the coroutine frame becomes more flexible and it should be brought more in line with `generator`'s allocator use. This paper addresses US 254-385, US 253-386, US 255-384, and US 261-391.

§ 1 Change History

§ 1.1 R0 Initial Revision

§ 2 Overview of Changes

There are a few NB comments about `task`'s use of allocators:

— US 254-385: Constraint `allocator_arg` argument to be the first argument

— US 253-386: Allow use of arbitrary allocators for coroutine frame

— US 255-384: Use allocator from receiver's environment

— US 261-391: Bad specification of parameter type

The first issue (US 254-385) is about where an allocator argument for the coroutine frame may go on the coroutine function. The options are a fixed location (which would fit first for consistency with existing use) and anywhere. The status quo is anywhere, and the request is to require that it goes first. However, to support optionally passing an allocator, having it go anywhere is easier to do.

The allocator constraints for allocating the coroutine frame are due to the use of the same allocator for the environment of child senders. If the allocator for the environment of child senders uses the allocator from the receiver's environment, these constraints can be relaxed. Instead, there may be requirements on the result of the `get_allocator` query from the receiver's environment. The discussion in Kona favored this direction. This change can address the second (US 253-386) and the third (US 255-384) issues.

The fourth issue (US 261-391) is primarily a wording issue. However, some of the problematic paragraphs will need some modifications to address the other issues, i.e., fixing these wording issues in isolation isn't reasonable.

§ 3 Allocator Argument Position

The combination of using `allocator_arg` followed by an allocator object when invoking a function or a constructor is used in various places throughout the standard C++ library. The `allocator_arg` argument normally needs to be the first argument when present. The definition of `task` makes the position of the `allocator_arg` more flexible to allow easier support for optionally passing an allocator.

For coroutines, the arguments to the coroutine [factory] function show up in three separate places:

1. The parameters to the coroutine [factory] functions.
2. The constructor of the `promise_type` if there is a suitable matching overload.
3. the `operator new()` of the `promise_type` if there is a suitable matching overload.

This added flexibility doesn't introduce any constraints on how the coroutine function is defined. It rather allows passing an `allocator_arg`/allocator pair without requiring a specific location. The main benefit is that support of an optional allocator can be supported by

having a trailing `, auto&&...` on the parameter list. Note that the allocator used for the coroutine frame is normally not used in the body of the allocator. If it is needed, it can in all cases be put into the first location.

First	Flexible
`task<> none(int x)` `{ ... }`	`task<> none(int x)` `{ ... }`

`task<> comes_first(allocator_arg_t,`
`  auto a, int x)`
`{ ... }`
	`task<> comes_first(allocator_arg_t,`
	`  auto a, int x)`
	`{ ... }`

`task<> optional(allocator_arg_t,`
`  auto, int x)`
`{ ... }`
`task<> optional(int x)`
`{ return optional(allocator_arg,`
`  allocator<char>(), x); }`
	`task<> optional(int x, auto&&...)`
	`{ ... }`

The comparison table above shows three separate cases the author of a coroutine function may want to support:

1. No allocator support (`none`): the use identical and just doesn't mention any allocator.
2. Mandate that the allocator is the first argument (`comes_first`): the use is identical.
3. Support optionally passing an `allocator_arg`/allocator pair (`optional`): the use can be identical but it can also be simplied taking advantage of the flexible location.

Below are three variations of the wording changes, only one can be picked:

A. Only support `allocator_arg` as the first argument and use the receiver's allocator for the environment.

B. Flexible position of the allocator arg and use allocator for the environment.

C. Flexible position of the allocator arg and use the receiver's allocator for the environment.

At the LEWG meeting on 2026-02-03 the first approach (putting the `allocator_arg` first, Wording Change A) was preferred (notes). It was identified that the original wording change did not support member functions returning a `task` (the wording was fixed accordingly).

§ 3.1 Wording Change A: `allocator_arg` must be first argument

[Editor's note: Change the synopsis of `promise` type in [task.promise], modifying the overloads of `operator new`:]

```
namespace std::execution {
  template<class T, class Environment>
  class task<T, Environment>::promise_type {
  public:
    ...
    unspecified get_env() const noexcept;

    void* operator new(size_t size);
    template<class Alloc, class... Args>
              void*  operator  new(size_t  size,  allocator_arg_t,  Alloc
        alloc,Args&&... ~~args~~);
    template<class This, class Alloc, class … Args>
       void* operator new(size_t size, const This&, allocator_arg_t, Alloc
         alloc, Args&&…);

    void operator delete(void* pointer, size_t size) noexcept;

  private:
    ...
  };
}
```

[Editor's note: Change [task.promise] paragraphs 17 and 18:]

```
void* operator new(size_t size);
```

?? *Returns*: `operator new(size, allocator_arg, allocator<byte>());`

```
template<class Alloc, class... Args>
   void* operator new(size_t size, allocator_arg_t, Alloc alloc,Args&&...
         ~~args~~);
template<class This, class Alloc, class … Args>
   void* operator new(size_t size, const This&, allocator_arg_t, Alloc al-
         loc, Args&&…);
```

17 ~~If there is no parameter with type `allocator_arg_t` then let `alloc` be `allocator_type()`. Otherwise, let `arg_next` be the parameter following the first `allocator_arg_t` parameter, and let `alloc` be `allocator_type(arg_next)`~~. Let `PAlloc` be `allocator_traits<~~allocator_type~~Alloc>::template rebind_alloc<U>`, where `U` is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

18 *Mandates*:

(18.1) — The first parameter of type `allocator_arg_t` (if any) is not the last parameter.

(18.2) — `allocator_type(arg_next)` is a valid expression if there is a parameter of type `allocator_arg_t`.

(18.3) — `allocator_traits<PAlloc>::pointer` is a pointer type.

18 *Mandates*: `allocator_traits<PAlloc>::pointer` is a pointer type.

19 *Effects*: Initializes an allocator `palloc` of type `PAlloc` with `alloc`. Uses `palloc` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `palloc`.

20 *Returns*: A pointer to the allocated storage.

§ 3.2 Wording Change B: Fix type names, allow flexible position, use for env

[Editor's note: Change [task.promise] pargraph 17 and 18 to use the correct type:]

```
template<class... Args>
  void* operator new(size_t size, Args&&... args);
```

17 If there is no parameter with type ~~`allocator_arg_t`~~`const allocator_arg_t&` then let `alloc` be `allocator_type()`. Otherwise, let `arg_next` be the parameter following the first ~~`allocator_arg_t`~~`const allocator_arg_t&` parameter, and let `alloc` be `allocator_type(arg_next)`. Let `PAlloc` be `allocator_traits<allocator_type>::template rebind_alloc<U>`, where `U` is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

18 *Mandates*:

(18.1) — The first parameter of type ~~`allocator_arg_t`~~`const allocator_arg_t&` (if any) is not the last parameter.

(18.2) — `allocator_type(arg_next)` is a valid expression if there is a parameter of type `allocator_arg_t`.

(18.3) — `allocator_traits<PAlloc>::pointer` is a pointer type.

19 *Effects*: Initializes an allocator `palloc` of type `PAlloc` with `alloc`. Uses `palloc` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `palloc`.

20 *Returns*: A pointer to the allocated storage.

§ 3.3 Wording Change C: Fix type names, allow flexible position, don't use for env

[Editor's note: Change [task.promise] pargraph 17 and 18 to use the correct type and don't convert to `allocator_type`:]

```
template<class... Args>
  void* operator new(size_t size, Args&&... args);
```

17 If there is no parameter with type ~~`allocator_arg_t`~~`const allocator_arg_t&` then let `alloc` be ~~`allocator_type()`~~`allocator()`. Otherwise, let ~~`arg_next`~~`alloc` be the parameter following the first ~~`allocator_arg_t`~~`const allocator_arg_t&` parameter~~, and let alloc be allocator_type(arg_next)~~. Let `PAlloc` be `allocator_traits<`~~`allocator_type`~~`remove_cvref_t<decltype(alloc)>>::template rebind_alloc<U>`, where `U` is an unspecified type whose size and alignment are both `__STDCPP_DEFAULT_NEW_ALIGNMENT__`.

18 *Mandates*:

(18.1) — The first parameter of type ~~`allocator_arg_t`~~`const allocator_arg_t&` (if any) is not the last parameter.

(18.2) — `allocator_type(arg_next)` is a valid expression if there is a parameter of type `allocator_arg_t`.

(18.3) — `allocator_traits<PAlloc>::pointer` is a pointer type.

19 *Effects*: Initializes an allocator `palloc` of type `PAlloc` with `alloc`. Uses `palloc` to allocate storage for the smallest array of `U` sufficient to provide storage for a coroutine state of size `size`, and unspecified additional state necessary to ensure that `operator delete` can later deallocate this memory block with an allocator equal to `palloc`.

20 *Returns*: A pointer to the allocated storage.

§ 4 Use Allocator From Environment

During the discussion at Kona the conclusion was that the allocator forwarded by `task`'s environment to child senders should be the allocator from `get_allocator` on the receiver `task` gets `connect`ed to. Let `rcvr` be the receiver a `task` got `connect`ed to and let `ev` be the result of `get_env(rcvr)`. The implication is that the `task`'s `allocator_type` is compatible with the allocator of `ev`:

- If `get_allocator(ev)` is not defined, `allocator_type` has to be default constructible.
- Otherwise `allocator_type(get_allocator(ev))` has to be well-formed.
- There is no need to store an *`alloc`* in the `promise_type`: it can be obtained when requested from `ev` which, in turn, can be obtained from `rcvr`. Thus, the ctor for `promise_type` isn't needed.
- The definition of `get_env` needs to be changed to get the allocator when needed.

§ 4.1 Wording Changes

[Editor's note: In [task.members] add a *Mandates* to `connect`:]

```
template<receiver Rcvr>
  state<Rcvr> connect(Rcvr&& recv) &&;
```

? *Mandates*: `allocator_type(get_allocator(get_env(rcvr)))` is well-formed or `allocator_type()` is well-formed.

3 *Preconditions*: `bool(handle)` is `true`.

4 *Effects*: Equivalent to:

```
return state<Rcvr>(exchange(handle, {}), std::forward<Rcvr>(recv));
```

[Editor's note: In [task.promise] in the synopsis remove the `promise_type` constructor and the *`alloc`* exposition-only member.]

```
namespace std::execution {
  template<class T, class Environment>
  class task<T, Environment>::promise_type {
  public:
    template<class… Args>
      promise_type(const Args&… args);

    task get_return_object() noexcept;

    ...
  private:
    using error-variant = see below;    // exposition only

    allocator_type    alloc;            // exposition only
    stop_source_type  source;           // exposition only
    stop_token_type   token;            // exposition only
    optional<T>        result;            // exposition only; present only
        if is_void_v<T> is false
    error-variant     errors;           // exposition only
  };
}
```

[Editor's note: Remove the ctor for `promise_type`, i.e., [task.promise] paragraph 3 and 4:]

```
template<class... Args>
  promise_type(const Args&... args);
```

3 *Mandates*: The first parameter of type `allocator_arg_t` (if any) is not the last parameter.

4 *Effects*: If `Args` contains an element of type `allocator_arg_t` then *`alloc`* is initialized with the corresponding next element of `args`. Otherwise, *`alloc`* is initialized with `allocator_type()`.

[Editor's note: Change `get_env` to get the allocator from the receiver when needed in [task.promise] p16:]

```
unspecified get_env() const noexcept;
```

16 *Returns*: An object `env` such that queries are forwarded as follows:

(16.1) — `env.query(get_scheduler)` returns `scheduler_type(`*`SCHED`*`(*this))`.

(16.2) — `env.query(get_allocator)` returns ~~`allo-`~~ ~~`c`~~`allocator_type(get_allocator(get_env(`*`RCVR`*`(*this))))` if this expression is well-formed and `allocator_type()` otherwise.

(16.3) — `env.query(get_stop_token)` returns *`token`*.

(16.4) — For any other query `q` and arguments `a...` a call to `env.query(q, a...)` returns *`STATE`*`(*this).environment.query(q, a…)` if this expression is well-formed and `forwarding_query(q)` is well-formed and is `true`. Otherwise `env.query(q, a...)` is ill-formed.